Building Dynamic, Intelligent Web Based Solutions with ASP.NET

I T W P 2 3 0 0

Paul C. Sbraccia, MA

Macomb Community College

2012

Version 4.0 Jan 2021

Table of Contents

First Day Activities **1**

Homework 1 ..... **11**

Homework 2 ..... **27**

Homework 3 ..... **37**

Homework 4 ..... **43**

Homework 5 ..... **55**

An Online eBook Book Store

Project 1 ..... **71**

Project 2 ..... **85**

Project 3 ..... **95**

Project 4 ..... **107**

Project 5 ..... **117**

ITWP 2300

BUILDING DYNAMIC, INTELLIGENT WEB BASED SOLUTIONS WITH ASP.NET

PROJECT 5

In this unit we will mine our data and include a second method of increasing sales. We will add a feature to the admin page to email customers when a new book is added by an author you purchased in the past.

ITWP 2300 eBook Store Project 5

Monday Morning Standup

Boss starts the meeting:

Wow! I cannot believe the progress your team has made on this BBN project. We had a meet-show-and-tell with Mr. Bob and he is very pleased with one small exception. He does not understand what all these error messages mean when he knows he has best sellers already created and just what that means.

I know we need lblErrorMessage as part of our App_Code interaction with the database, but do we need to actually show it?

TEAM: No, we just need it. We can set the visible property to false throughout the project so the code can use it, but the user won't see it.

BOSS: Good.

Now for the good news! All we need to do now is create the XML file containing the email of each person who ever bought a book written by an author that we are adding.

```
<? xml version = "1.0" encoding = "utf-8" ?>
<EMAIL>
<RECORD>
<AUTHOR>MAXINE PAETRO AND JAMES PATTERSON</AUTHOR>
<TITLE>10TH ANNIVERSARY</TITLE>
<IMAGEURL>10THANNIVERSARY.JPG</IMAGEURL>
<ADDRESS>myPC@S.com </ADDRESS >
</ RECORD >
<RECORD>
<AUTHOR>MAXINE PAETRO AND JAMES PATTERSON</AUTHOR>
<TITLE>10TH ANNIVERSARY</TITLE>
<IMAGEURL>10THANNIVERSARY.JPG</IMAGEURL>
<ADDRESS>pcs@yahoo.com </ADDRESS >
</ RECORD >
<RECORD>
<AUTHOR>MAXINE PAETRO AND JAMES PATTERSON</AUTHOR>
<TITLE>10TH ANNIVERSARY</TITLE>
<IMAGEURL>10THANNIVERSARY.JPG</IMAGEURL>
<ADDRESS>psbraccia@yahoo.com </ADDRESS >
</ RECORD >
<RECORD>
<AUTHOR>MAXINE PAETRO AND JAMES PATTERSON</AUTHOR>
<TITLE>10TH ANNIVERSARY</TITLE>
<IMAGEURL>10THANNIVERSARY.JPG</IMAGEURL>
<ADDRESS>QBall@yahoo.com </ADDRESS >
</ RECORD >
```

Ed Note: Sample XML file required by the email program, for which you are NOT responsible. We do have to create the XML file. We do NOT have to export it or save it anywhere for the purposes of Project 5.

Here is how we envision this email piece working:

When a new XML file is received from the publisher, we understand how to upload the images and process the XML file to add the new information into our database. We will invoke this with yet another button on Admin.aspx. By the way, Bob likes buttons. ;)

Fortunately, the XML file stays on the server until the next XML file is received and processed. We can use this to our advantage and start with a list of newly added book authors, then check History to see who purchased any books previously by that author, and create the email.XML file shown above. How you do it, is up to you. I have this SQL command from the Database Administrator who says it is the "Cat's Meow". Fortunately, I have you guys to make that stuff work.

Did I mention the production bonus if this is finished by the end of December? Remember our mandatory shutdown period in your planning.

ITWP 2300 eBook Store Project 5

Project Prep

Copy BobsBookNook4 over to BobsBookNook5.

Admin.aspx

Start by adding yet another (ahhh the last) button to **Admin.aspx** that points to **ProcessEmail.aspx**. **Email Marketing** (or something appropriate) is ok for the text.

Ed Note: There is no need for error checking because if we get this far, the bookInventory.xml file HAS to exist.

Before we go on to creating ProcessEmail.aspx, let's be sure to search for lblErrorMessage throughout the entire project and set the visible property to false. (You may have already done this.)

Ed Note: One way to do this is open each aspx file in Design mode, highlight the lblErrorMessage (there might be some slightly different), and set the visible property to false.

ProcessEmail.aspx

Copy **ProcessEmail.aspx** from the templates folder.

Ed Note: Once you test the Admin button that calls this page, you can set this page as the default page for testing and development.

Be sure to set the <title> text to **Process Email**.
Ed Note: You can use the code from processBestsellers to populate txtOwnerID.Text .

Then add the following
:

- Button: btnBackToAdmin
- TextBox: txtOwnerID
- Label: lblErrorMessage Visible Property set to false.
- ListBox: lstRawXML
- ListBox: lstEmail

Double check that following using statements have been included.

- **using System.IO;**
- **using System.Data.SqlClient;**
- **using System.Data;**

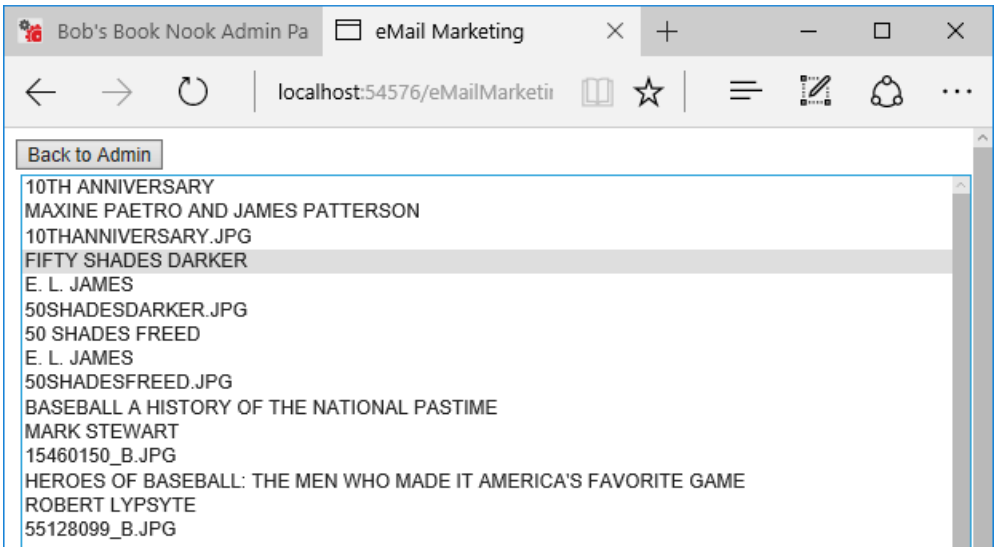
The initial code in ProcessEmail.aspx borrows code from Admin.aspx to read the inventory xml file.

We examine each line and only process those lines we are interested in: Title, Author and Image.

```
protected void Page_Load(…
{

    try
    {
       var data = File.ReadAllText(Server.MapPath("~/App_Data/ownerID.txt"));
      txtOwnerID.Text = data.ToString().Trim();
    } // end try
    catch
    {
      Response.Redirect("Login.aspx");
    }

   string fileName = Server.MapPath("~/img") + "\\BookInventory.xml";
   try
   {
       if (File.Exists(fileName))
       using (StreamReader sr = new StreamReader(fileName))
       {
           while (sr.Peek() >= 0)
            {
               string lineIn = sr.ReadLine();
               lineIn = lineIn.Trim().ToUpper();
               if (lineIn.StartsWith("<TITLE>") || lineIn.StartsWith("<AUTHOR>")
|| lineIn.StartsWith("<IMAGE>"))

                   {
                       lineIn = stripTags(lineIn);
                       lstRawXML.Items.Add(lineIn);
                   }
            }
       }
   } // end try
   catch (Exception f)
   {
       Console.WriteLine("The process failed: {0}", f.ToString());
   }
}
```

stripTags is missing. ☺ Again, borrowed from Admin.aspx.

```
private string stripTags(string field)
{
    //remove up to first >
    int x = field.IndexOf(">");
    field = field.Substring(x + 1);
    //remove second tag
    x = field.LastIndexOf("<");
    field = field.Substring(0, x);
    return (field);
}
```



Running the program at this point displays Title, Author, then Image for each "new" text.

Before we continue, we need to add a btnGo.

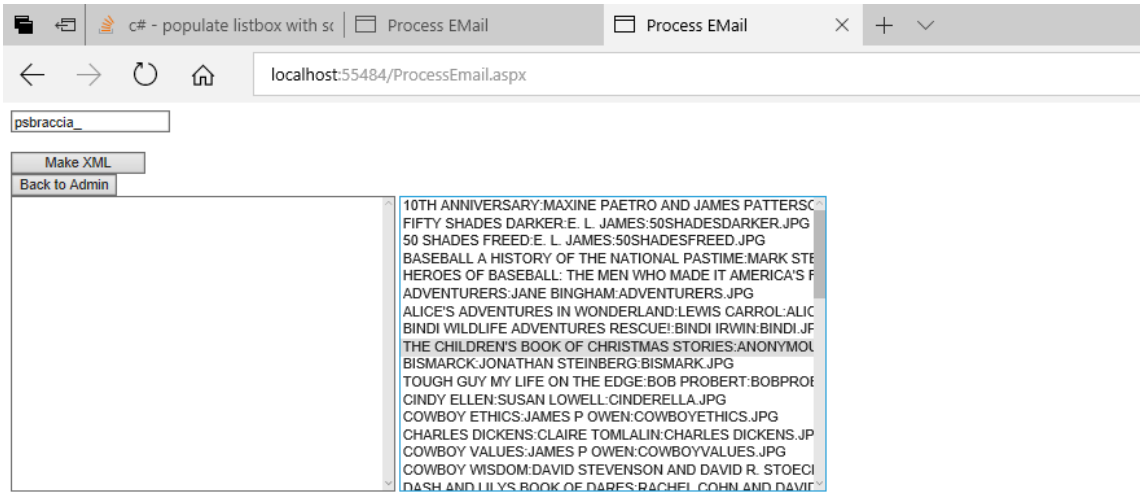
We start by placing the Title, Author and Image as a record in lstEmail. We then use Author to search BOOKS for other books by the Author. Once we identify other books by the author, we look in History to see who may have purchased one or more identified ISBNs and report the email address.

```
protected void btnGo_Click(…
{
string sqlCommand;
string author;
string title;
string image;
while (lstRawXML.Items.Count > 0)
  {
      //title
      title = lstRawXML.Items[0].ToString();
      lstRawXML.Items.RemoveAt(0);
      //author
      author = lstRawXML.Items[0].ToString();
      lstRawXML.Items.RemoveAt(0);
      //image
      image = lstRawXML.Items[0].ToString();
      lstRawXML.Items.RemoveAt(0);
      //add the new record to lstEmail
      lstEMail.Items.Add(title + ":" + author + ":" + image);
      //look for users this is that "cat's meow" sql
      sqlCommand = "SELECT DISTINCT " + txtOwnerID.Text + "HISTORY.EMAIL FROM " +
txtOwnerID.Text + "HISTORY WHERE " + txtOwnerID.Text + "HISTORY.ISBN IN (SELECT DISTINCT
" + txtOwnerID.Text + "BOOKS.ISBN FROM " + txtOwnerID.Text + "BOOKS WHERE " +
txtOwnerID.Text + "BOOKS.AUTHOR = '" + author + "')";
      fillListBox(sqlCommand, lstEmail,  txtOwnerID.Text + "HISTORY", "EMAIL");
      btn2.Enabled = true;
      btnGo.Enabled = false;
  }
}
```

Unfortunately there is the fact that we need to create fillListBox.

```
void fillListBox(string sqlCommand, ListBox listBox, string tableName, string field)
{
	DataRow dr;
	DataTable dt = new DataTable();
	try
	{ //open the connection
		myDatabaseConnection.myConnection.Open();
		SqlDataAdapter da = new SqlDataAdapter(sqlCommand,
myDatabaseConnection.myConnection);
		da.Fill(dt);
		for (int i = 0; i < dt.Rows.Count; i++)
		{
			dr = dt.Rows[i];
			listBox.Items.Add(dr[field].ToString());
		}
		listBox.SelectedIndex = 0;
	}
	catch (Exception ex) { lblErrorMessage.Text = ex.ToString(); }
	try
	{ //close the connection
		myDatabaseConnection.myConnection.Close();
	}
	catch (Exception ex) { lblErrorMessage.Text = ex.ToString(); }
}
```

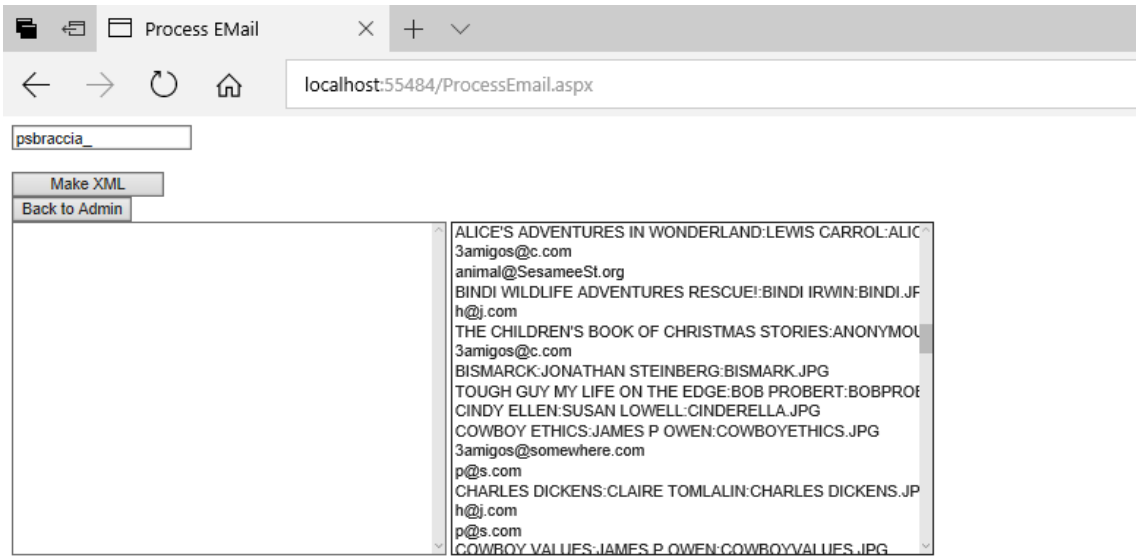
As you review the code above you will recognize that it is very similar to fillDropDownList that is already in App_Code.
The reason we place it here rather than in myDatabaseConnection is this is a single use [as well as for a List, not a DropDown Box] and very specialized, thus not ready for prime time sharing.



With the SQL statement commented out.

Ed Note: You can always move this function to myDatabaseConnection later.

We need to use dr and dt in order to pull individual items from a SQL query placing the item in a list or other non gridview object. If this looks familiar, we used similar code to load dropdown lists in Ashland Soccer.



Note that we have a record containing the title, author and image delimited by :'s followed by email addresses. It is good that **lstRawXML** is empty as we are going to use it as the output created by makeXML.

XML

Looking at what we need in the XML file:

```
<RECORD>
<AUTHOR>MAXINE PAETRO AND JAMES PATTERSON</AUTHOR>
<TITLE>10TH ANNIVERSARY</TITLE>
<IMAGEURL>10THANNIVERSARY.JPG</IMAGEURL>
<ADDRESS>myPC@S.com </ADDRESS >
</ RECORD >
```

Apparently, we will need to create a <record></record> for each email address in the list that contains <AUTHOR>, <TITLE>, <IMAGEURL> and <ADDRESS>,

Add a button called **btnStep2**. Set the enabled attribute to false.

At the bottom of btnGo code, add the following two lines of code. The purpose is to prevent the user from pressing the second button too early.

```
btnStep2.Enabled = true;
btnGo.Enabled = false;
```

```
protected void btnStep2_Click(…
{
    string[] record = null;

    //throw new NotImplementedException();
    //Clear lstRawXML
    lstRawXML.Items.Clear();
    //Add the first two lines of the XML file
    lstRawXML.Items.Add("<? xml version = \"1.0\" encoding = \"utf-8\" ?>");
    lstRawXML.Items.Add("<EMAIL>");
    // process each record in the dropdown list
    while (lstEmail.Items.Count > 0)
    {
        if (lstEmail.Items[0].ToString().Contains(":"))
        {
            //get and parse title, author, image
            record = lstEmail.Items[0].ToString().Split(':');
        }
        else
        {
            lstRawXML.Items.Add("<RECORD>");
            lstRawXML.Items.Add("<AUTHOR>" + record[1] + "</AUTHOR>");
            lstRawXML.Items.Add("<TITLE>" + record[0] + "</TITLE>");
            lstRawXML.Items.Add("<IMAGEURL>" + record[2] + "</IMAGEURL>");
            lstRawXML.Items.Add("<ADDRESS>" + lstEmail.Items[0].ToString() + "</ADDRESS > ");
            lstRawXML.Items.Add(" </ RECORD > ");
        }
       //remove the first element in the dropdown list
        lstEmail.Items.RemoveAt(0);
    }
    //When done, add the closing </EMAIL> tag.
    lstRawXML.Items.Add("</EMAIL>");
}
```

Make sure **lblErrorMessage.Visible** = **false**.

- Make sure that your entire app works as it is intended.
- Make sure all lblErrorMessage.Visible attributes are set to false throughout the entire project.
- Use the new BookInventory.xml to load new books and execute eMail Marketing.
- You are permitted to change the button names to better explain to users the steps they need to take.